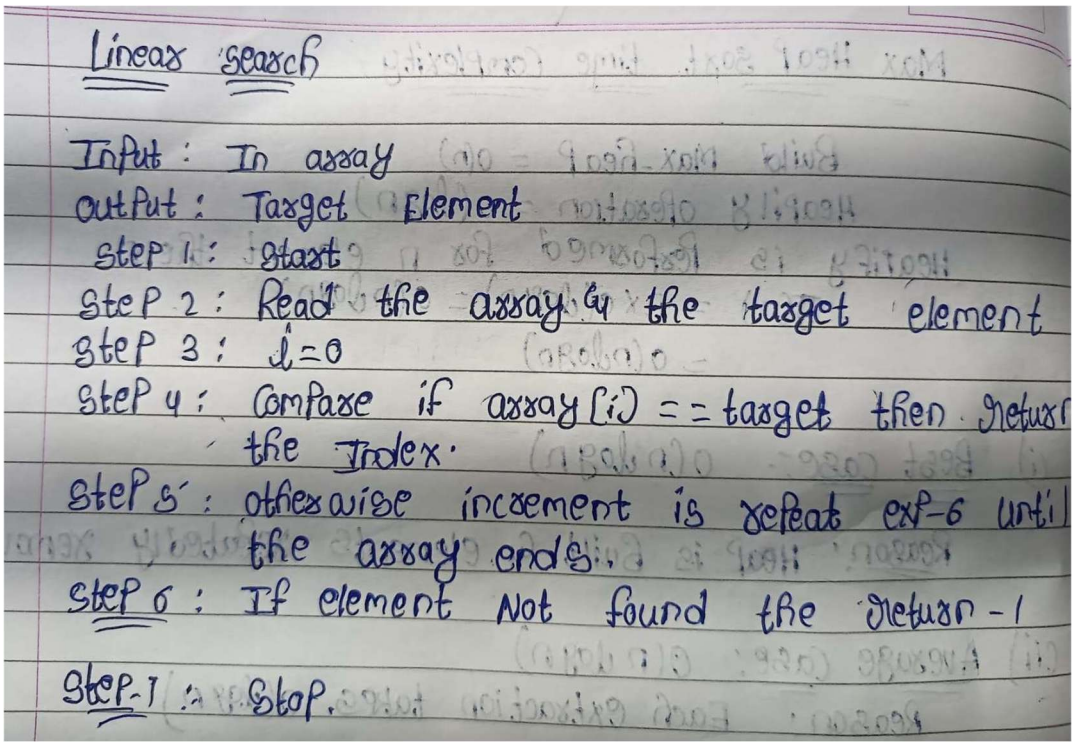
Practical: 2	Implementation and Time analysis of linear and binary search algorithm
05/08/2026	

Aim: To Implement Linear search and binary search and Time analysis their algorithms.

Linear search

Algorithm:



M Shankara rao (92400118007)

Program:

```
#include  <iostream>

using namespace std;

int main() {
   int n;
   cout << "Enter the number of elements: ";
   cin >> n;

   int arr[n];
   cout << "Enter " << n << " elements: ";
   for (int i = 0; i < n; i++) {
      cin >> arr[i];
   }

   int target;
   cout << "Enter the element to search: ";
   cin >> target;

   int index = -1;
   for (int i = 0; i < n; i++) {
      if (arr[i] == target) {
         index = i;
         break;
      }
   }

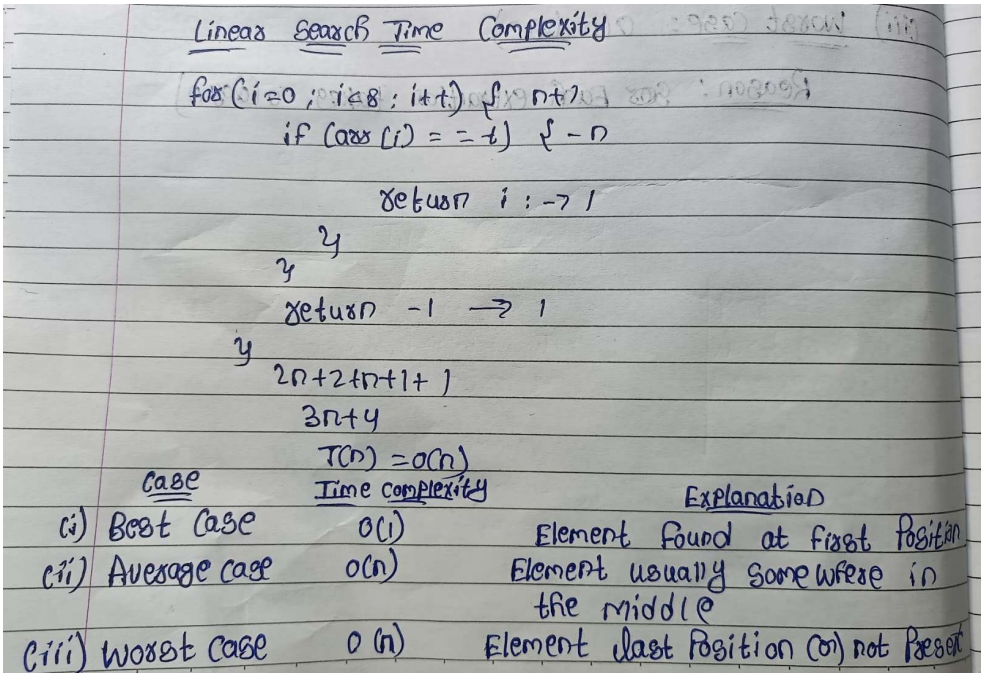
   if (index != -1) {
      cout << "Element found at index: " << index << endl;
   } else {
      cout << "Element not found" << endl;
   }

   return 0;
}
```

Output:

```
Enter the number of elements: 5
Enter 5 elements: 66
32
55
45
6
Enter the element to search: 6
Element found at index: 4
```

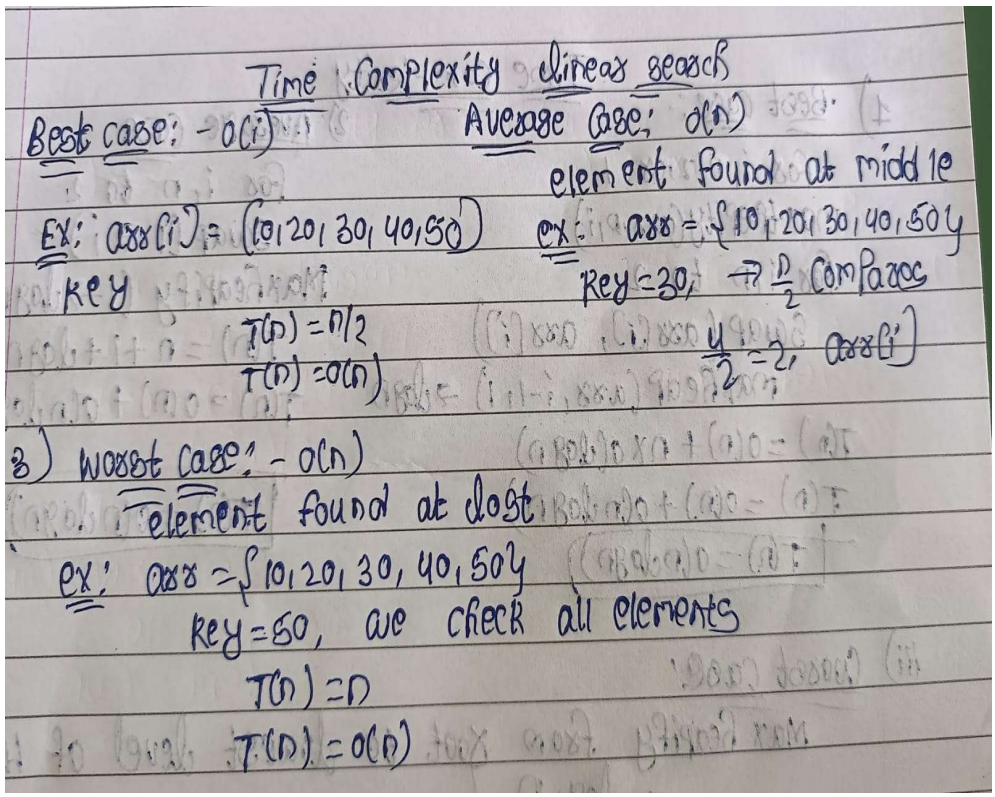
Time Complexity:



Linear Search Time Complexity

```
for(i=0; i<8; i++) {n+1
    if (arr[i] == -1) {-n
        return i; -> 1
    }
}
return -1 -> 1
}
2n+2+n+1+1
3n+4
T(n) = O(n)
```

Case	Time complexity	Explanation
(i) Best Case	O(1)	Element found at first position
(ii) Average case	O(n)	Element usually somewhere in the middle
(iii) Worst case	O(n)	Element last position (or) not Present



Time Complexity linear search

Best case: - O(1)

Ex: arr(i) = (10, 20, 30, 40, 50)
key

T(n) = n/2
T(n) = O(n)

Average Case: O(n)
element found at middle

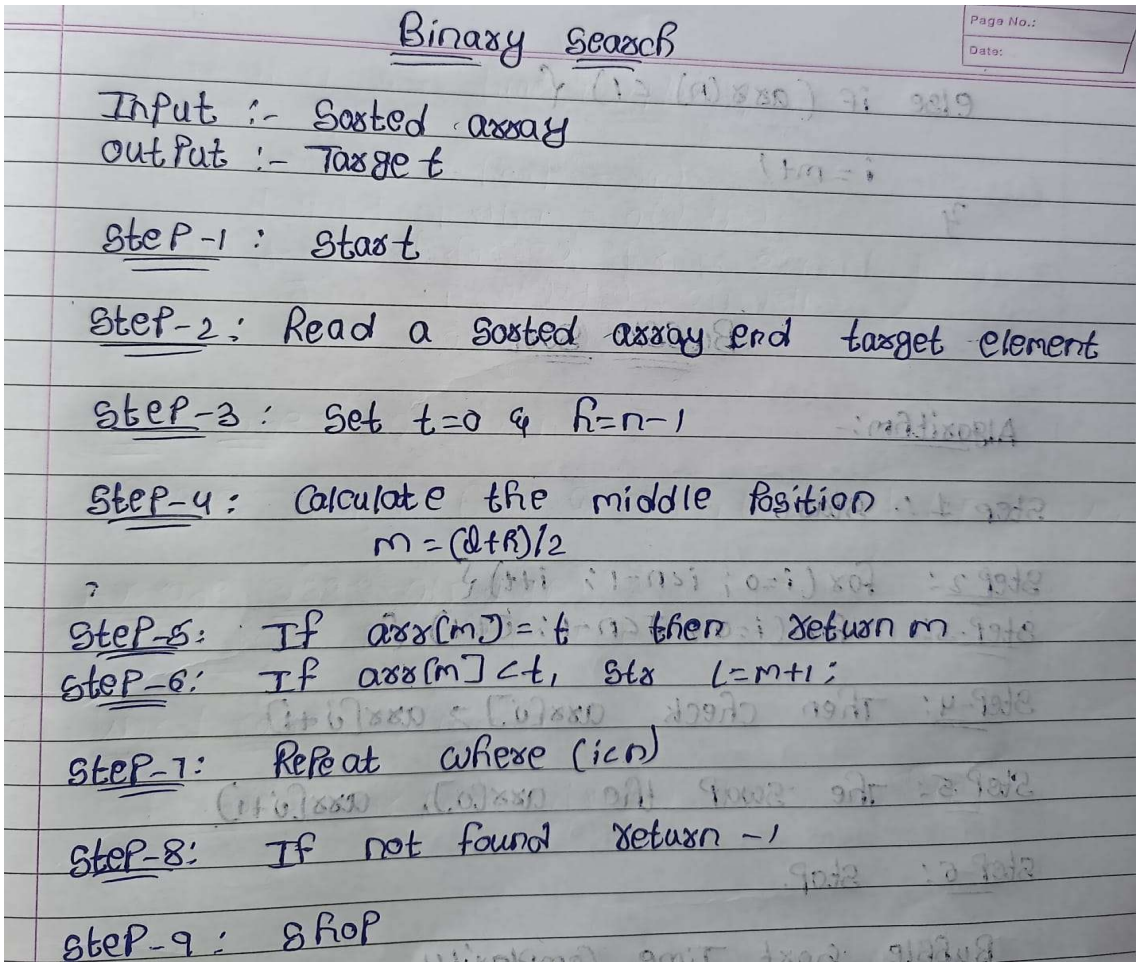
ex: arr = {10, 20, 30, 40, 50}
key = 30, -> n/2 Comparos
n/2 = 2, arr[i]

3) Worst case: - O(n)
element found at last

ex: arr = {10, 20, 30, 40, 50}
key = 50, we check all elements
T(n) = n
T(n) = O(n)

Binary search

Algorithm:



Program code:

```
#include <iostream>
#include <vector>

using namespace std;

int binarySearch(const vector<int>& arr, int target) {
    int low = 0;
    int high = arr.size() - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == target) {
            return mid;
        } else if (arr[mid] < target) {
```

```
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return -1;
}

int main()
```

```
{
    int size;
    cout << "Enter the number of elements: ";
    cin >> size;

    vector<int> arr(size);
    cout << "Enter " << size << " elements in sorted order:\n";
    for (int i = 0; i < size; i++) {
        cin >> arr[i];
    }

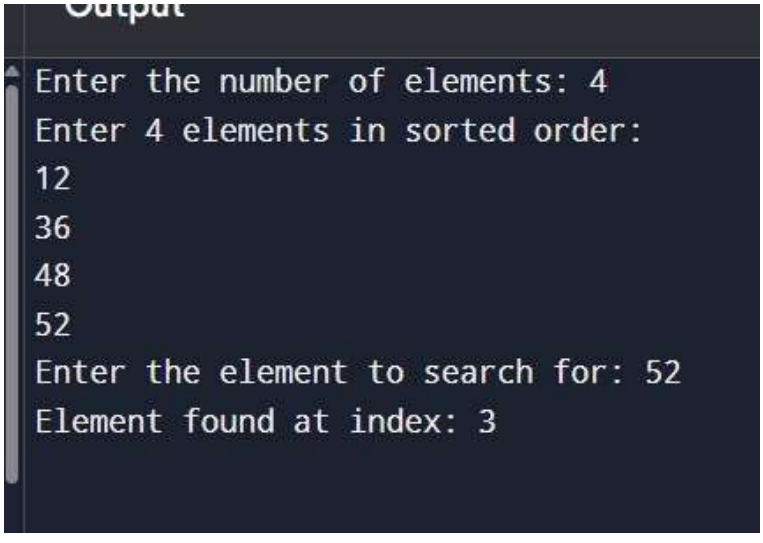
    int target;
    cout << "Enter the element to search for: ";
    cin >> target;

    int result = binarySearch(arr, target);

    if (result != -1) {
        cout << "Element found at index: " << result << endl;
    } else {
        cout << "Element not found in the array." << endl;
    }

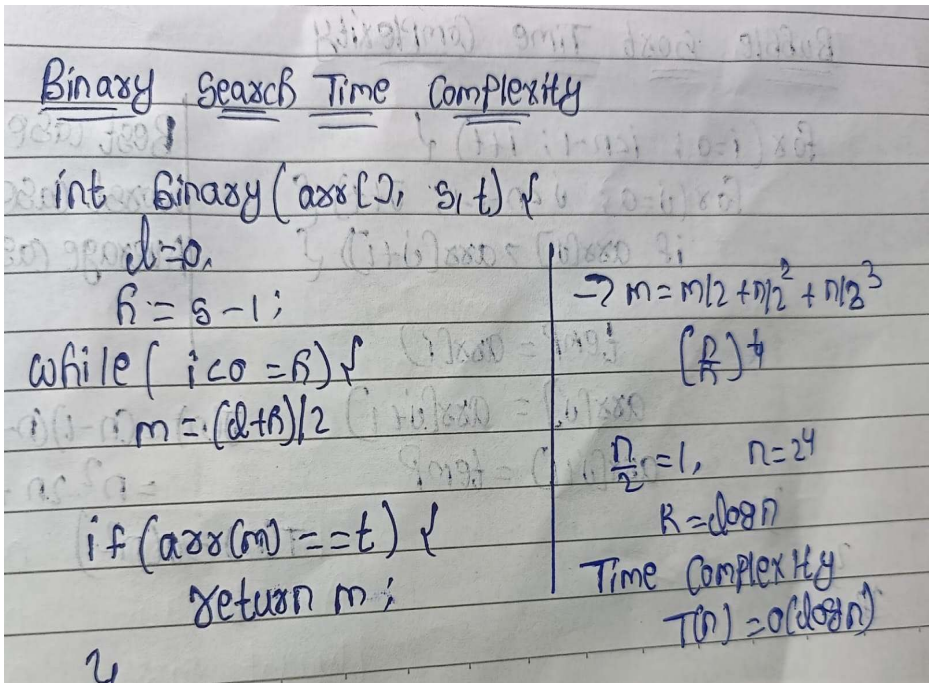
    return 0;
}
```

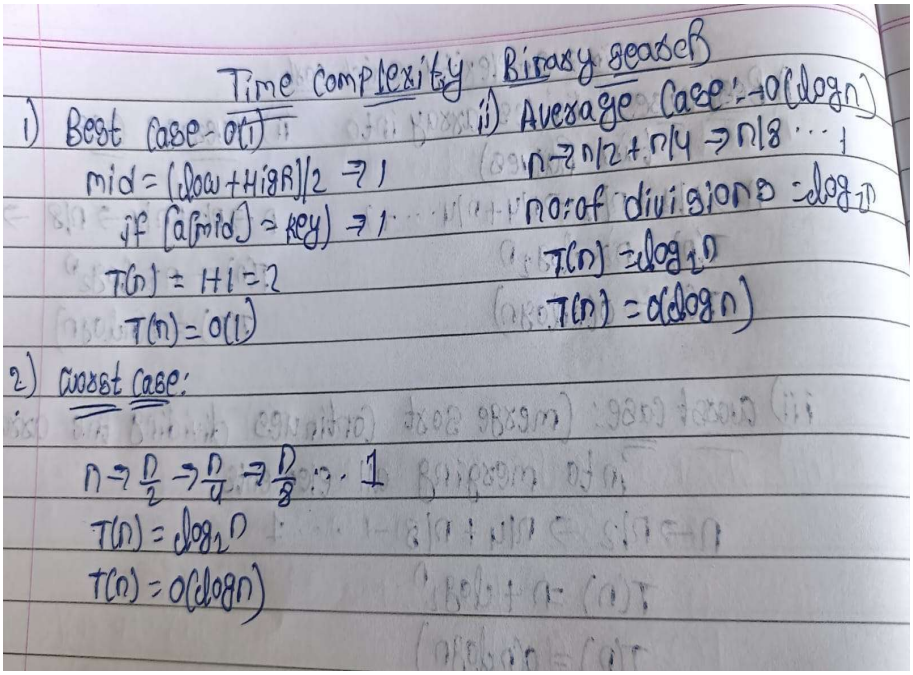
Output:



+

Time complexity:





Conclusion:
Linear search and binary search both are searching techniques used to find out an element to search An array.linear checks the element one by one works on both sorted and unsorted elemnts then then Elements of o(n) and 0(nlogn) are the complexities.